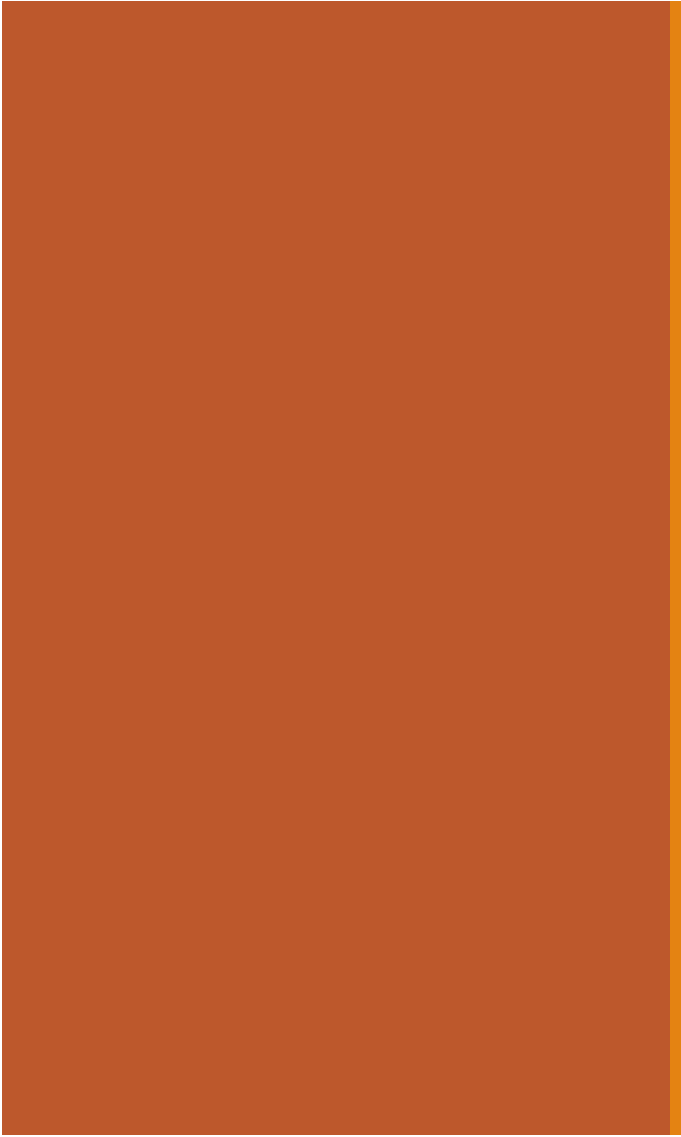


25: Logistic Regression

Jerry Cain
May 29, 2024

[Lecture Discussion on Ed](#)



Linear to Logical: Preamble

1. Weighted sum

Dot product: $\theta^T \mathbf{X} = \sum_{j=1}^m \theta_j X_j$

Recall the linear regression model, where $\mathbf{X} = (X_1, X_2, \dots, X_m)$ and $Y \in \mathbb{R}$:

$$g(\mathbf{X}) = \theta_0 + \sum_{j=1}^m \theta_j X_j$$

if we define Z such that
 $Z = \theta_1 X_1 + \theta_2 X_2 + \theta_3 X_3$
then it can be written more compactly
as $Z = \theta^T \mathbf{X}$, where

How would you rewrite this expression as a single dot product? $\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$ & $\mathbf{X} = \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix}$

$$g(\mathbf{X}) = \theta_0 X_0 + \theta_1 X_1 + \theta_2 X_2 + \dots + \theta_m X_m \quad \text{Define } X_0 = 1$$

$$= \theta^T \mathbf{X}$$

$$\text{New } \mathbf{X} = (1, X_1, X_2, \dots, X_m)$$

$$\text{and } \theta = (\theta_1, \theta_2, \dots, \theta_m)$$

Prepending $X_0 = 1$ to each feature vector $\mathbf{X}$
simplifies matrix operations.

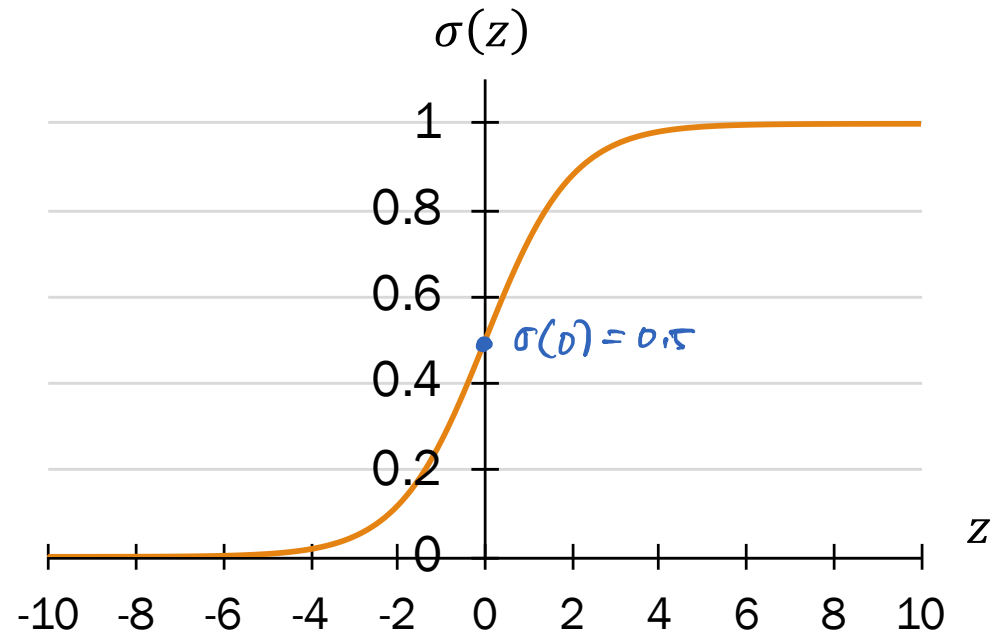


2. Sigmoid function $\sigma(z)$

- The sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Sigmoid squashes z to a number between 0 and 1.



- Recall definition of probability:
A number between 0 and 1 that expresses a belief that something is true.

$\sigma(z)$ can represent a probability.

3. Conditional likelihood function

Training data (n datapoints):

- $(\mathbf{x}^{(i)}, y^{(i)})$ drawn iid from a distribution $f(\mathbf{X} = \mathbf{x}^{(i)}, Y = y^{(i)} | \theta) = f(\mathbf{x}^{(i)}, y^{(i)} | \theta)$

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta)$$

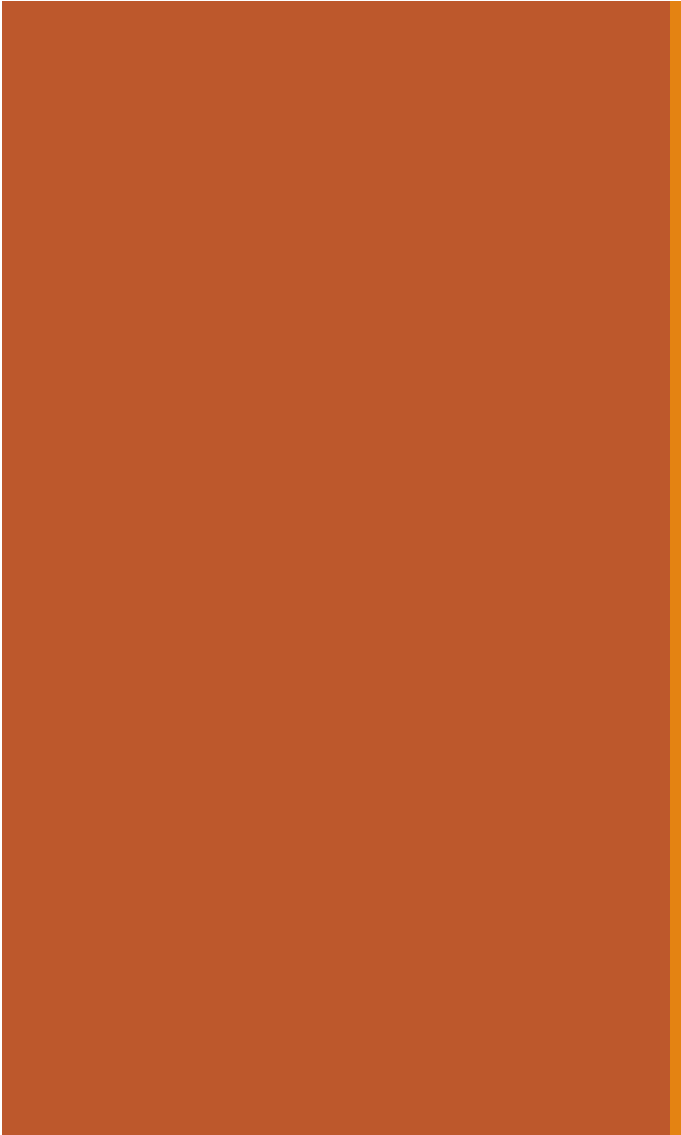
conditional likelihood
of training data

$$= \arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | \mathbf{x}^{(i)}, \theta)$$

log conditional likelihood

$$= \arg \max_{\theta} LL(\theta)$$

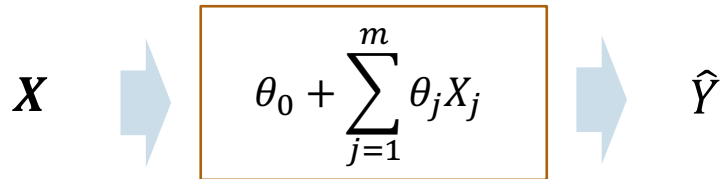
- MLE here is estimator that maximizes **conditional** likelihood
- log conditional likelihood is also written as $LL(\theta)$



Logistic Regression

Prediction models so far

Linear Regression (Regression)



$$\hat{Y} = \theta_0 + \sum_{j=1}^m \theta_j X_j$$

this is the

m-dimensional generalization of last Friday's $\hat{Y} = aX + b$

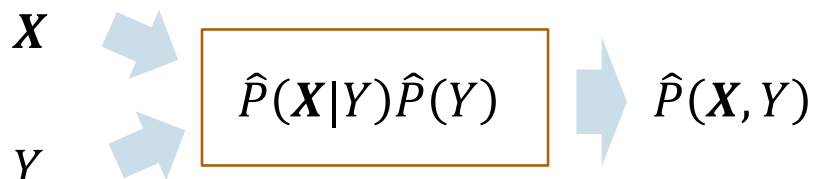


X can be dependent



Regression model ($\hat{Y} \in \mathbb{R}$, not discrete)

Naïve Bayes (Classification)



$$\begin{aligned}\hat{Y} &= \arg \max_{y=\{0,1\}} P(Y | X) \\ &= \arg \max_{y=\{0,1\}} P(X|Y)P(Y)\end{aligned}$$



Tractable with NB assumption, but...

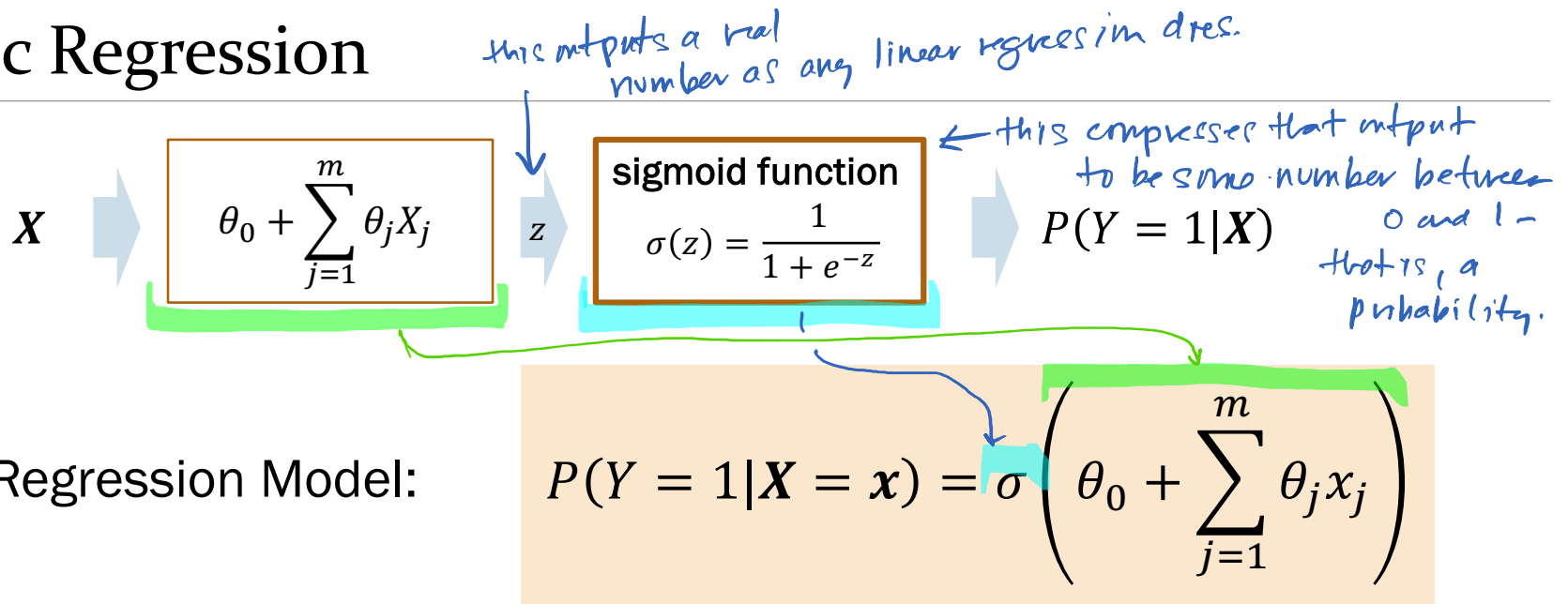


Realistically, X_j features aren't always conditionally independent



Actually models $P(X, Y)$, not $P(Y|X)$?

Logistic Regression

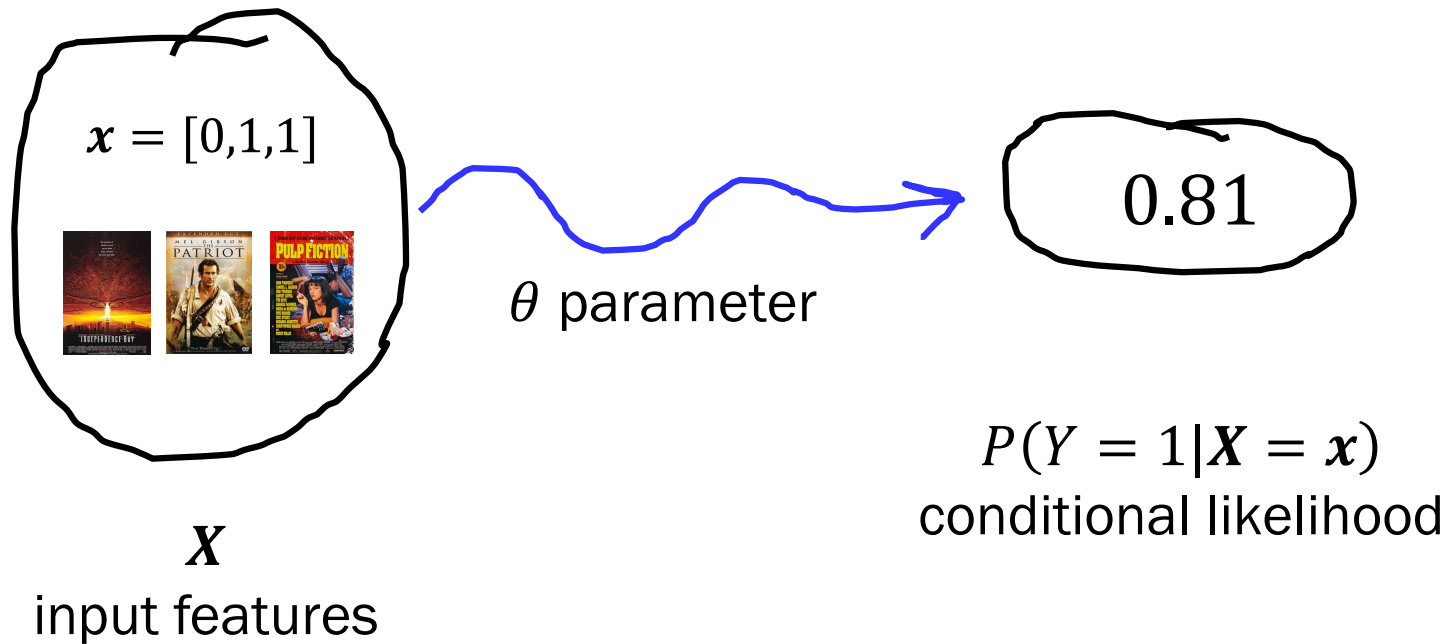


Predict $\hat{Y}$ as the more likely Y given our observation $X = x$:

$$\hat{Y} = \arg \max_{y \in \{0,1\}} P(Y | X)$$

- Since $Y \in \{0,1\}$, $P(Y = 0 | X = x) = 1 - \sigma(\theta_0 + \sum_{j=1}^m \theta_j x_j)$
- Sigmoid function also known as **logit function** $= 1 - P(Y=1 | X=x)$

Logistic Regression



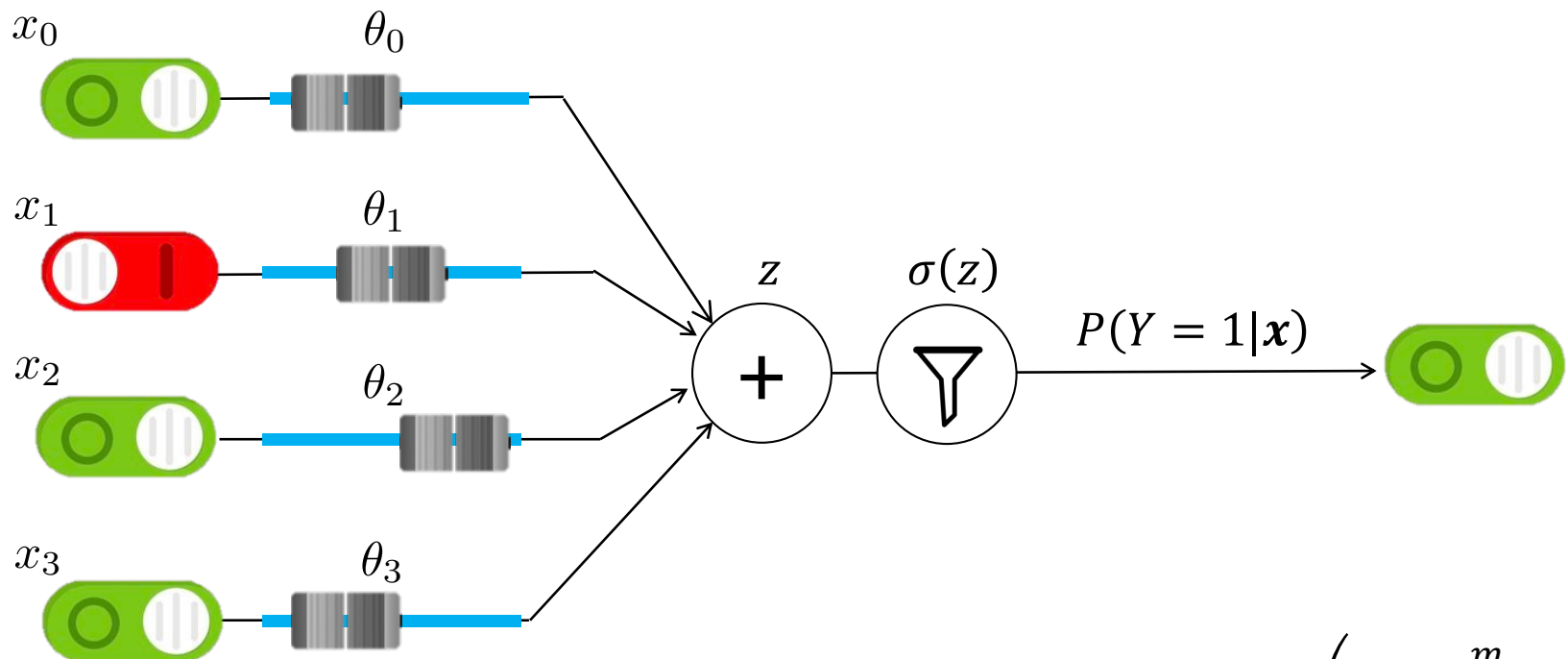
$$P(Y = 1 | X = x) = \sigma \left(\theta_0 + \sum_{j=1}^m \theta_j x_j \right)$$

Logistic Regression: Key Metaphor



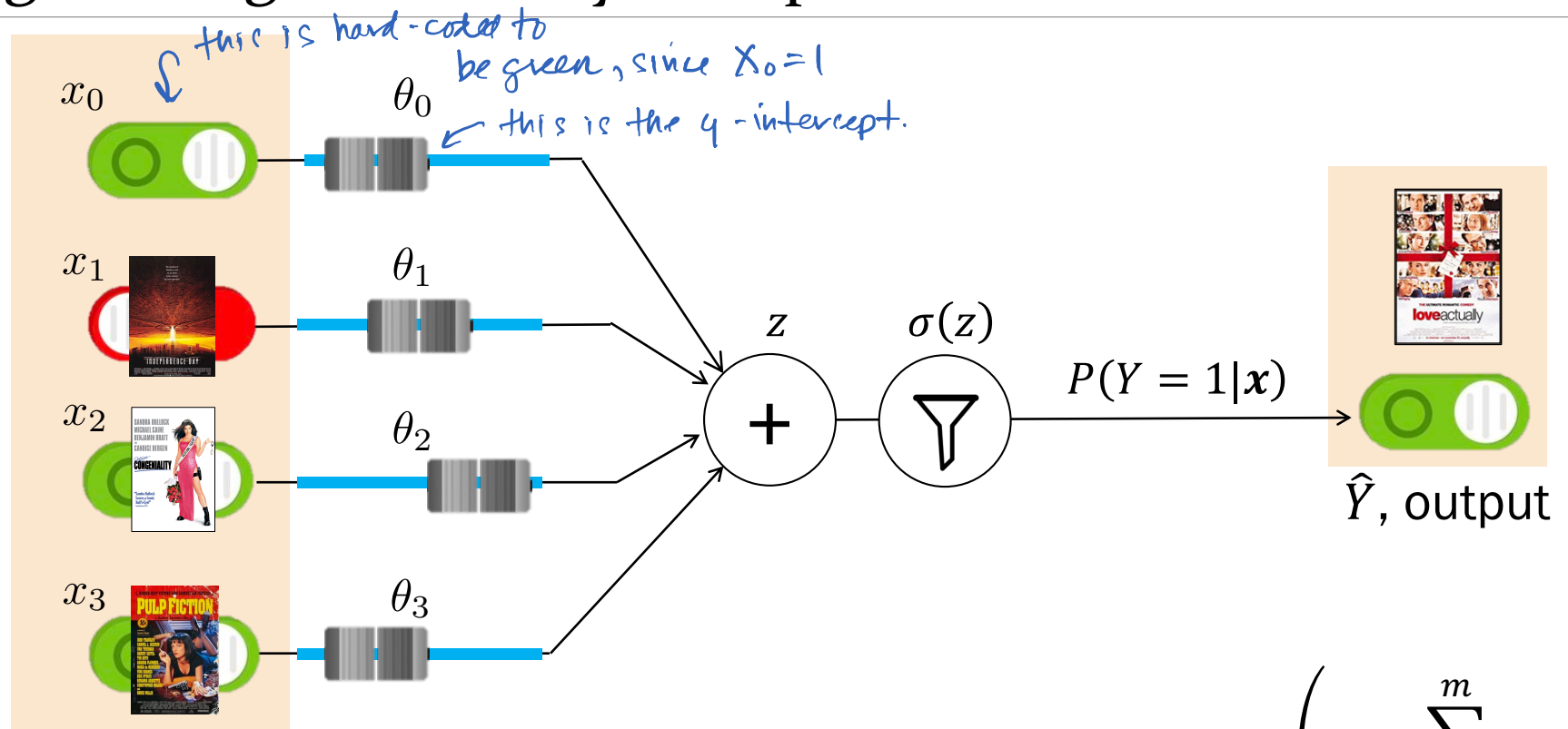
θ parameter

Logistic Regression: Key Metaphor



$$P(Y = 1 | \mathbf{X} = \mathbf{x}) = \sigma \left(\theta_0 + \sum_{j=1}^m \theta_j x_j \right)$$

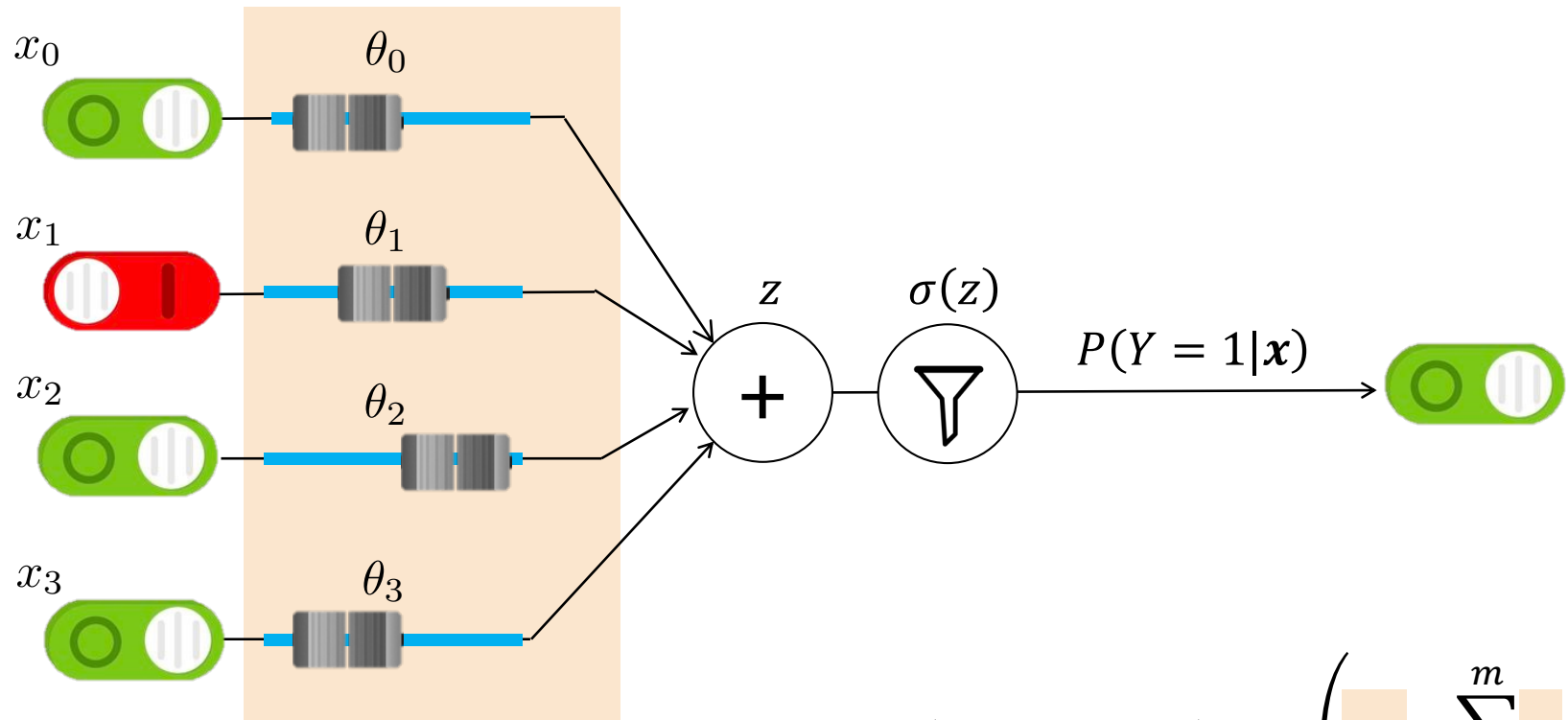
Logistic Regression: Key Metaphor



$\mathbf{X}$, input features
[0,1,1]

$$P(Y = 1 | \mathbf{X} = \mathbf{x}) = \sigma \left(\theta_0 + \sum_{j=1}^m \theta_j x_j \right)$$

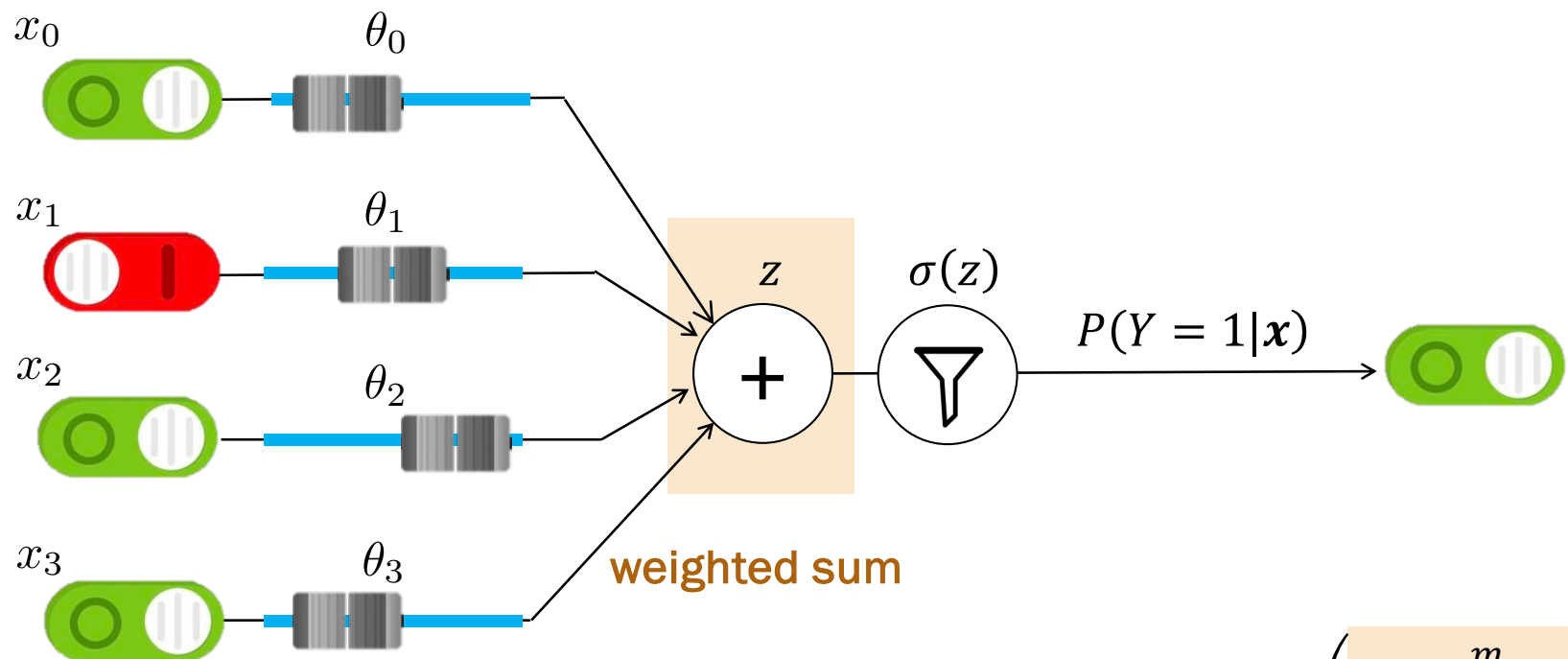
Components of Logistic Regression



θ weights
(aka parameters)

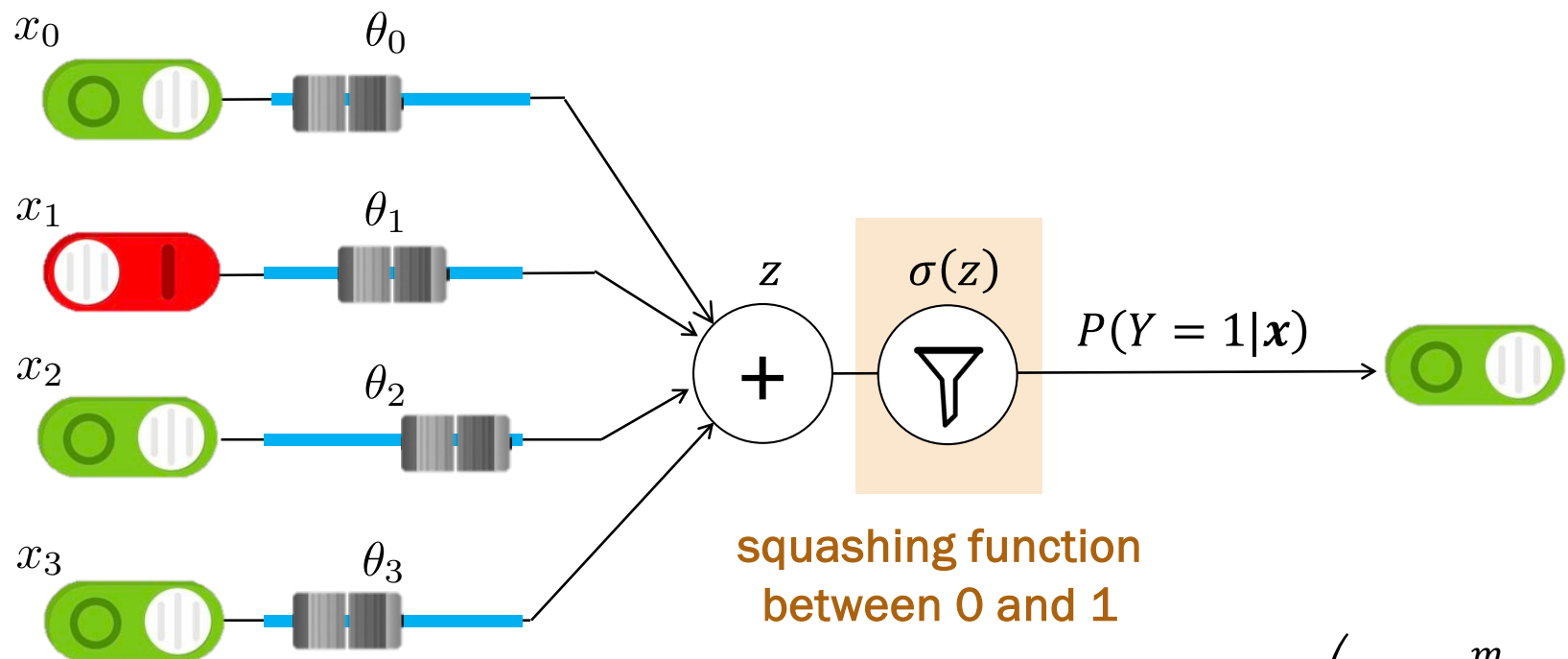
$$P(Y = 1|X = \mathbf{x}) = \sigma \left(\theta_0 + \sum_{j=1}^m \theta_j x_j \right)$$

Components of Logistic Regression



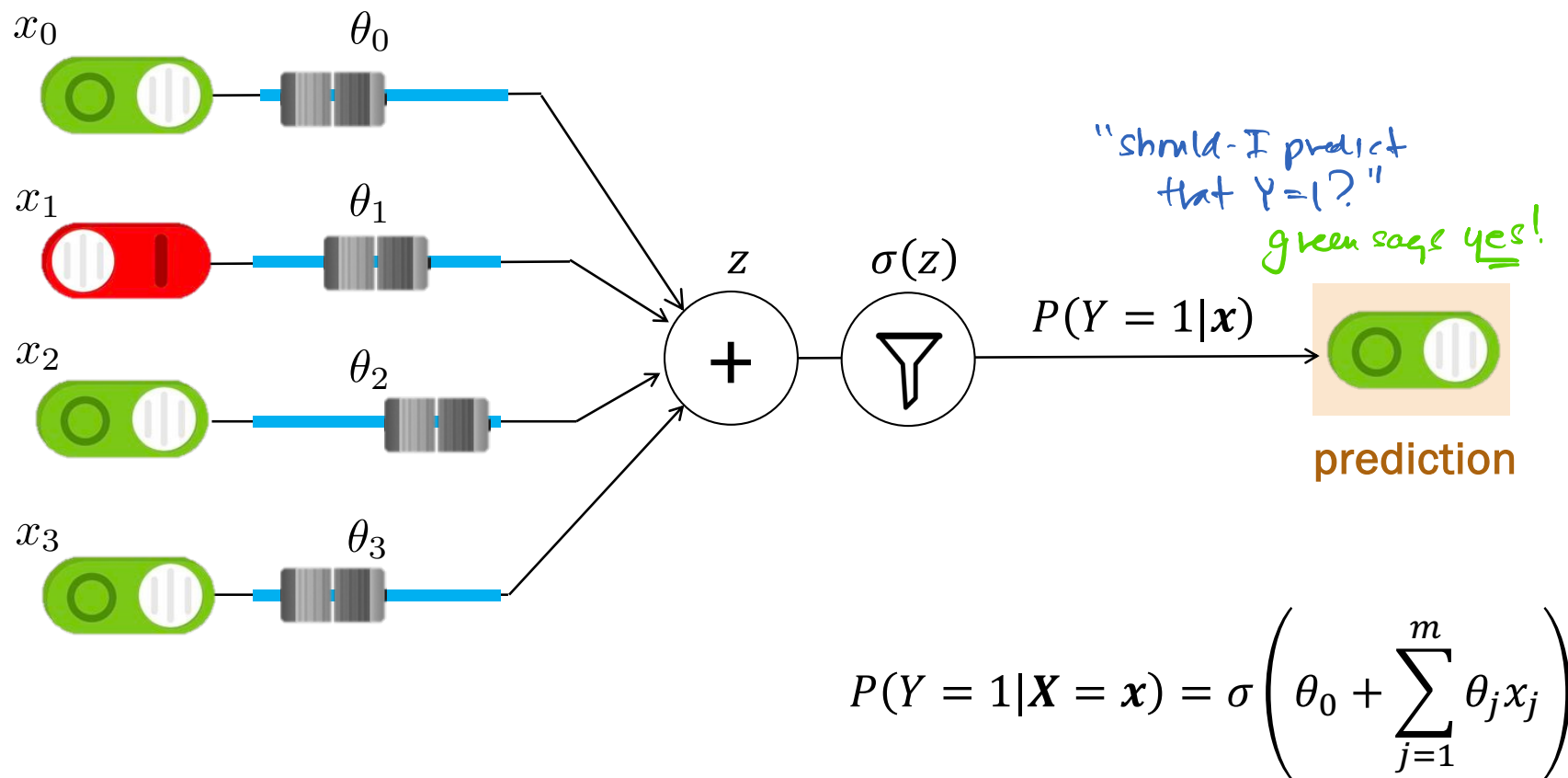
$$P(Y = 1|X = \mathbf{x}) = \sigma \left(\theta_0 + \sum_{j=1}^m \theta_j x_j \right)$$

Components of Logistic Regression

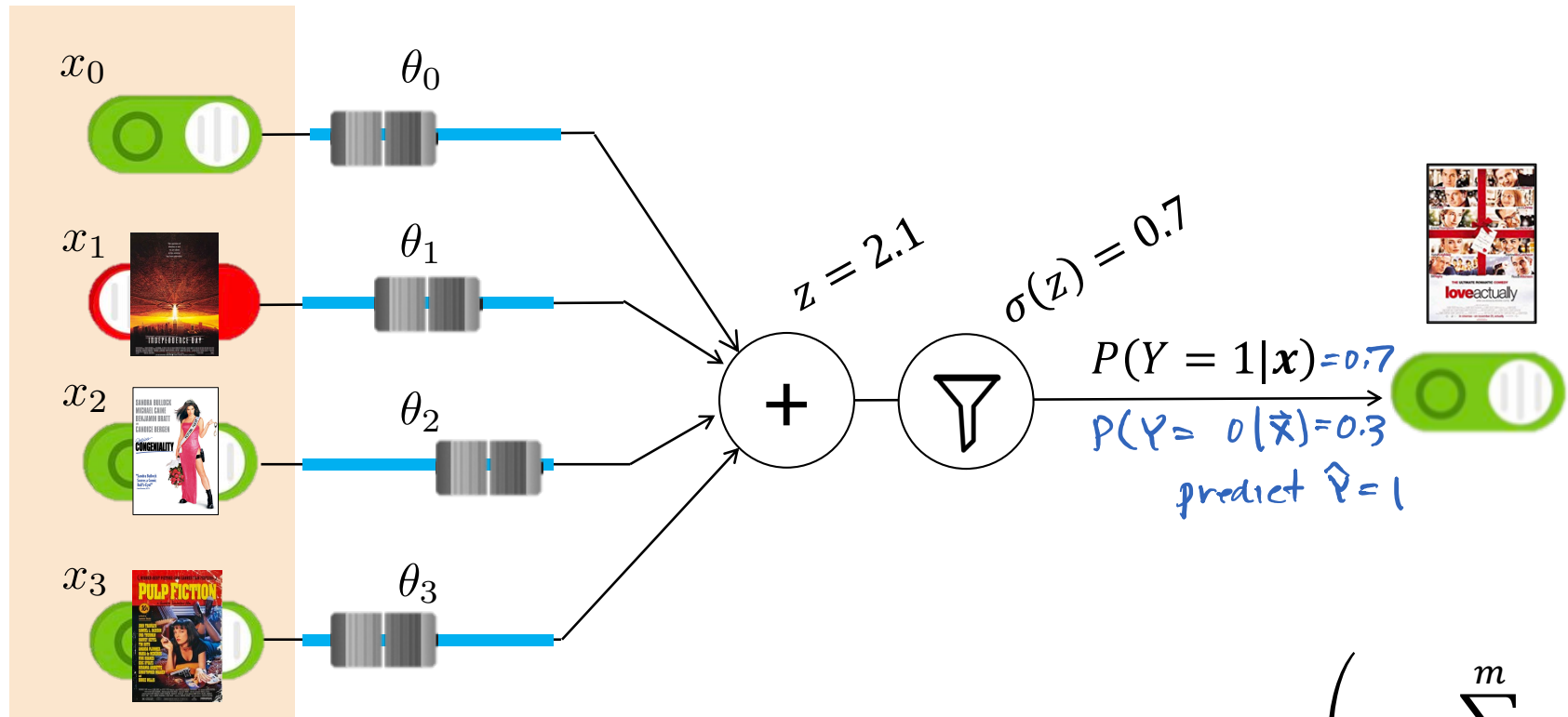


$$P(Y = 1 | \mathbf{X} = \mathbf{x}) = \sigma \left(\theta_0 + \sum_{j=1}^m \theta_j x_j \right)$$

Components of Logistic Regression



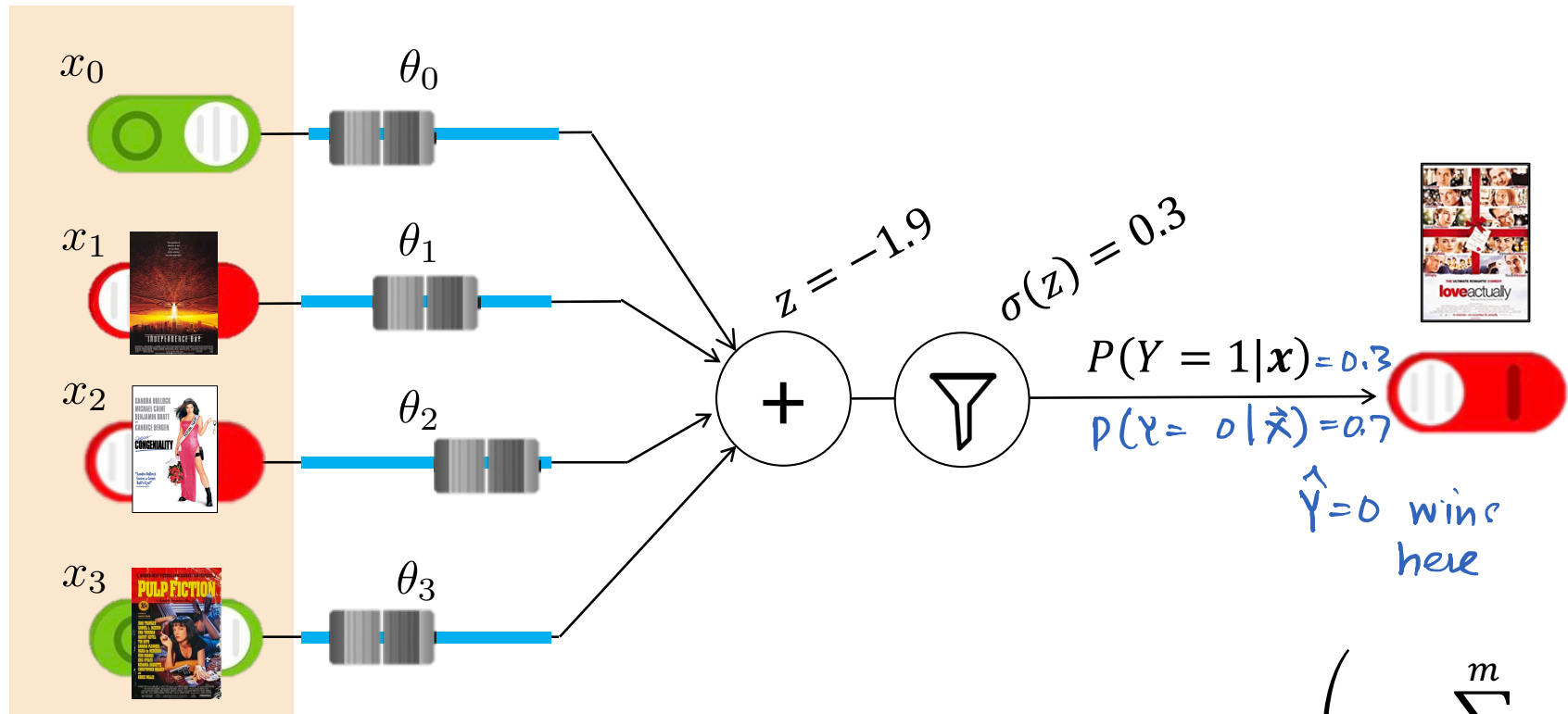
Different predictions for different inputs



$\mathbf{X}$, input features
[0,1,1]

$$P(Y = 1 | \mathbf{X} = \mathbf{x}) = \sigma \left(\theta_0 + \sum_{j=1}^m \theta_j x_j \right)$$

Different predictions for different inputs

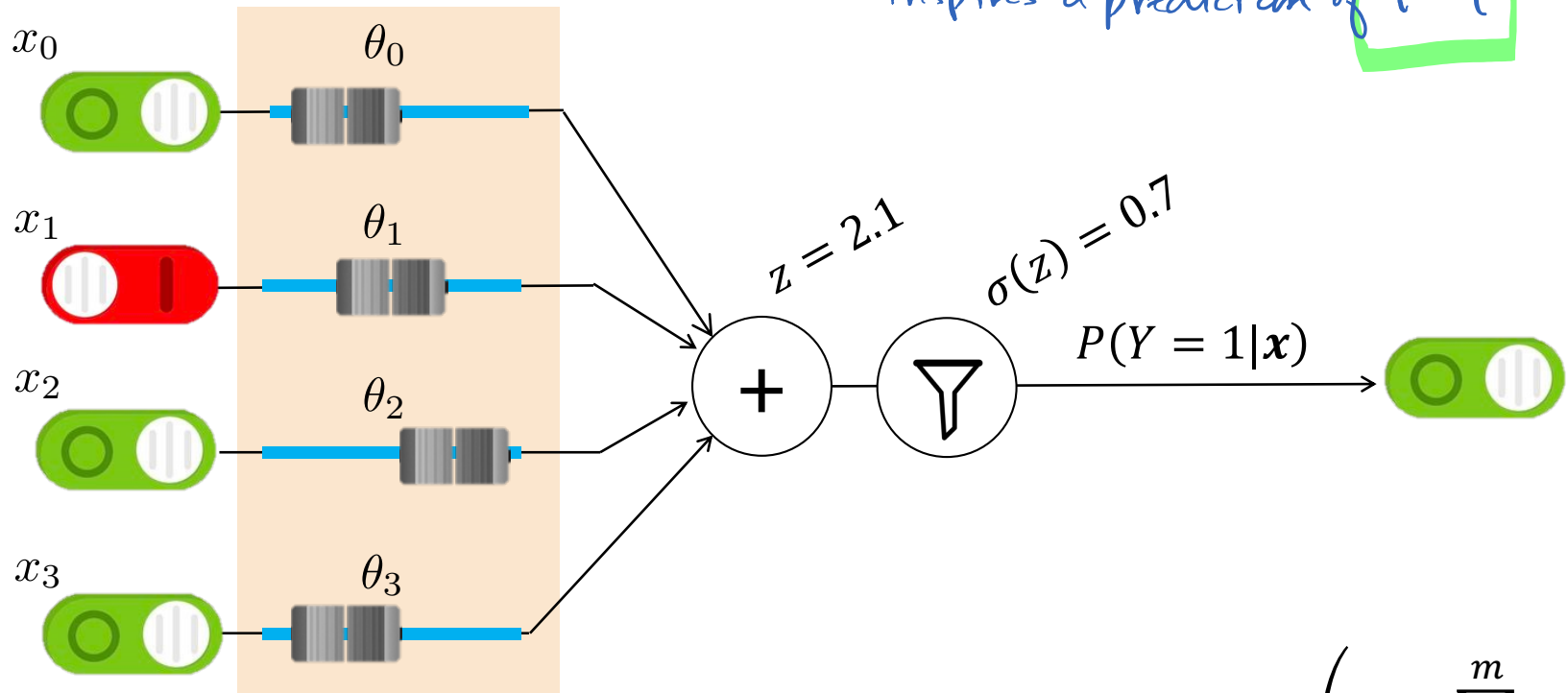


$\mathbf{X}$, input features
[0,0,1]

$$P(Y = 1 | \mathbf{X} = \mathbf{x}) = \sigma \left(\theta_0 + \sum_{j=1}^m \theta_j x_j \right)$$

Parameters affect prediction

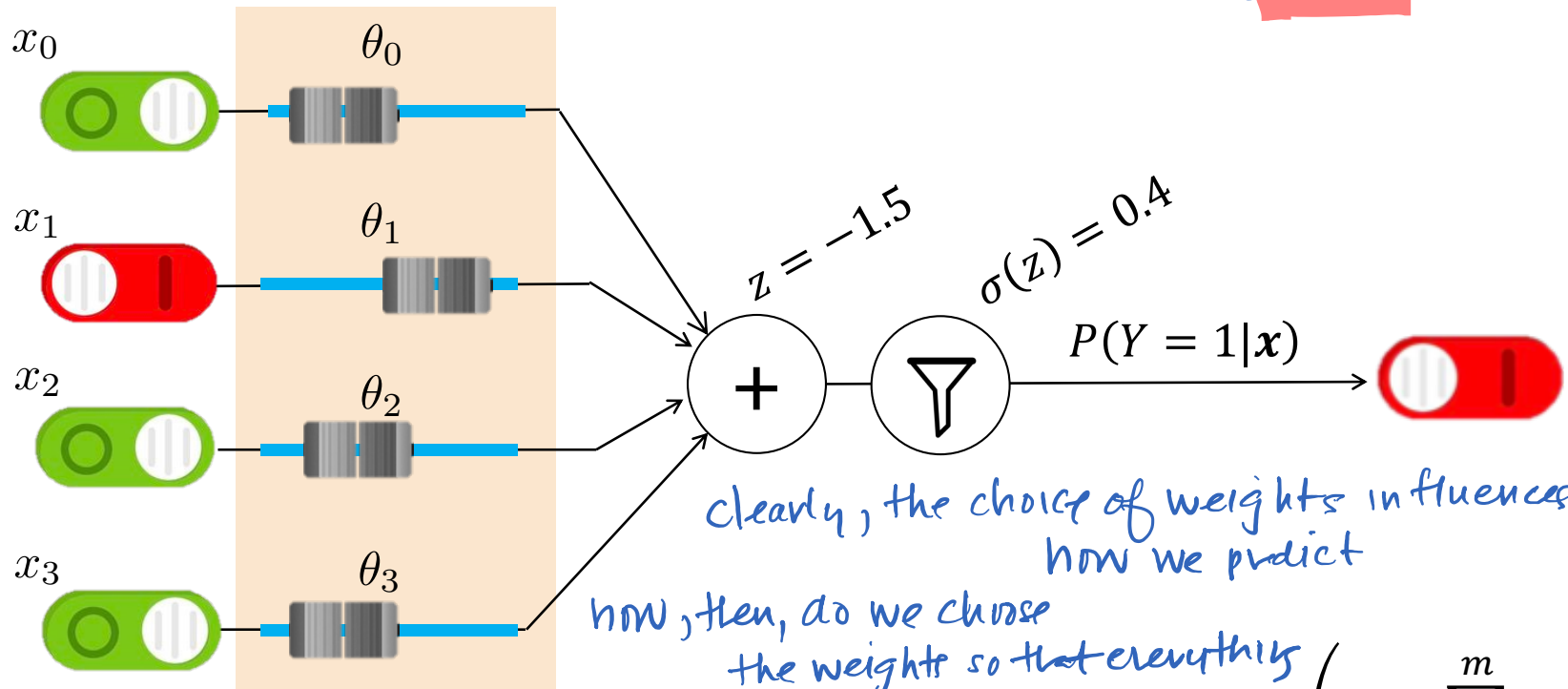
this particular set of weights
inspires a prediction of $\hat{y} = 1$



$$P(Y = 1|X = \mathbf{x}) = \sigma \left(\theta_0 + \sum_{j=1}^m \theta_j x_j \right)$$

Parameters affect prediction

this new set of weights inspire
a prediction of $\hat{Y}=0$



clearly, the choice of weights influences
how we predict

now, then, do we choose

the weights so that everything

$$P(Y = 1|X = x) = \sigma \left(\theta_0 + \sum_{j=1}^m \theta_j x_j \right)$$

In a training set is
accurately predicted?

Parameters affect prediction

$$P(Y = 1|X = \mathbf{x}) = \sigma\left(\theta_0 + \sum_{j=1}^m \theta_j x_j\right)$$



$$P(Y = 1|X = \mathbf{x}) = \sigma\left(\sum_{j=0}^m \theta_j x_j\right) = \sigma(\theta^T \mathbf{x}) \quad \text{where } x_0 = 1$$

Logistic regression classifier

$$\hat{Y} = \arg \max_{y=\{0,1\}} P(Y|\mathbf{X})$$

$$P(Y = 1|\mathbf{X} = \mathbf{x}) = \sigma\left(\sum_{j=0}^m \theta_j x_j\right) = \sigma(\theta^T \mathbf{x})$$

Training

Estimate parameters
from training data

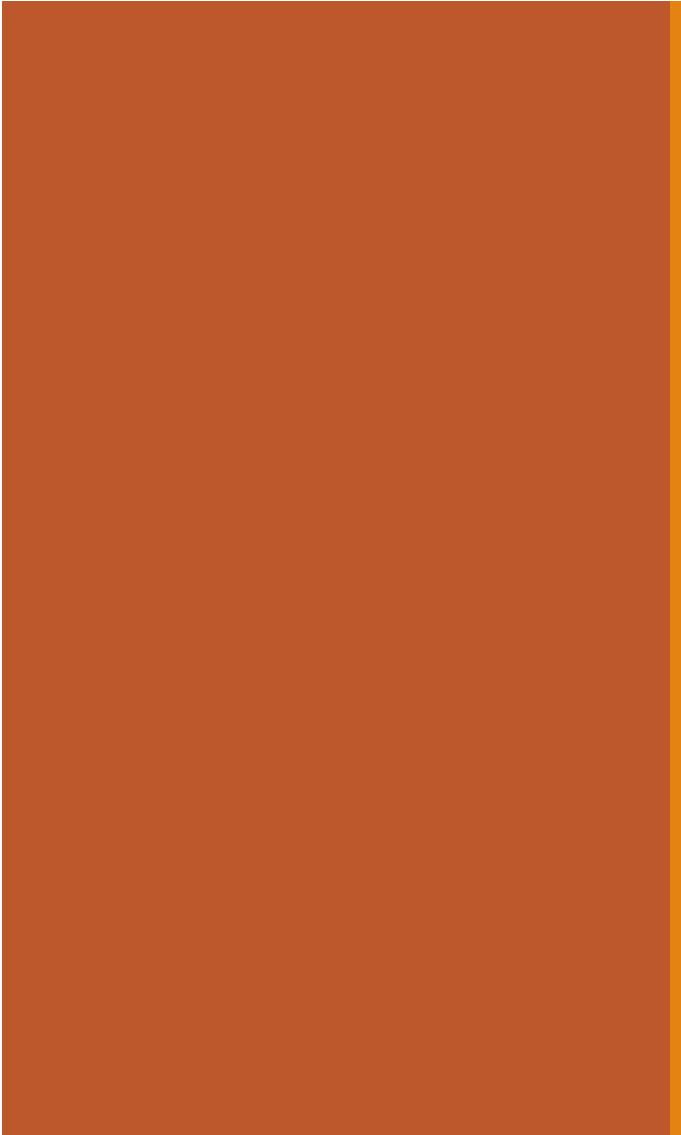
$$\theta = (\theta_0, \theta_1, \theta_2, \dots, \theta_m)$$

Testing

Given an observation $\mathbf{X} = (X_1, X_2, \dots, X_m)$, predict

$$\hat{Y} = \arg \max_{y=\{0,1\}} P(Y|\mathbf{X})$$

and how well does it match
the true output label?



Training: The big picture

Logistic regression classifier

$$\hat{Y} = \arg \max_{y=\{0,1\}} P(Y|\mathbf{X})$$

$$P(Y = 1|\mathbf{X} = \mathbf{x}) = \sigma\left(\sum_{j=0}^m \theta_j x_j\right) = \sigma(\theta^T \mathbf{x})$$

restated, compute probability that $Y=1$ and $Y=0$, and choose $\hat{Y}$ to be the value of Y that comes with the higher probability

Training

Estimate parameters from training data

$$\theta = (\theta_0, \theta_1, \theta_2, \dots, \theta_m)$$

Choose θ that optimizes some objective:

1. Determine objective function
2. Find gradient with respect to each θ (all m of them)
3. Solve analytically by setting to 0, or solve computationally with gradient ascent

We are modeling $P(Y|X)$ directly, so we maximize the **conditional likelihood** of training data.

Estimating θ

1. Determine objective function

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta)$$

2. Gradient wrt θ_j , for $j = 0, 1, \dots, m$

3. Solve

- No analytical derivation of θ_{MLE} ...
- ...but can still determine θ_{MLE} via gradient ascent!

```
initialize x
repeat many times:
  compute gradient
  x +=  $\eta$  * gradient
```

1. Determine objective function

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$
$$P(Y = 1 | \mathbf{X} = \mathbf{x}) = \sigma\left(\sum_{j=0}^m \theta_j x_j\right) = \sigma(\theta^T \mathbf{x})$$

First: Interpret
conditional likelihood
with Logistic Regression

Second: Write a differentiable
expression for log conditional
likelihood

1. Determine objective function (interpret)

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$

$$P(Y = 1 | X = \mathbf{x}) = \sigma(\sum_{j=0}^m \theta_j x_j) \\ = \sigma(\theta^T \mathbf{x})$$

Suppose you have $n = 2$ training datapoints: $(\mathbf{x}^{(1)}, 1), (\mathbf{x}^{(2)}, 0)$

Consider the following expressions for a given θ :

A. $\sigma(\theta^T \mathbf{x}^{(1)}) \sigma(\theta^T \mathbf{x}^{(2)})$
 $P(Y=1 | X=\mathbf{x}^{(1)}) P(Y=1 | X=\mathbf{x}^{(2)})$

B. $(1 - \sigma(\theta^T \mathbf{x}^{(1)})) \sigma(\theta^T \mathbf{x}^{(2)})$
 $P(Y=0 | X=\mathbf{x}^{(1)}) P(Y=1 | X=\mathbf{x}^{(2)})$

C. $\sigma(\theta^T \mathbf{x}^{(1)}) (1 - \sigma(\theta^T \mathbf{x}^{(2)}))$
 $P(Y=1 | X=\mathbf{x}^{(1)}) P(Y=0 | X=\mathbf{x}^{(2)})$

D. $(1 - \sigma(\theta^T \mathbf{x}^{(1)})) (1 - \sigma(\theta^T \mathbf{x}^{(2)}))$
 $P(Y=0 | X=\mathbf{x}^{(1)}) P(Y=0 | X=\mathbf{x}^{(2)})$

1. Interpret the above expressions as probabilities.

2. If we let $\theta = \theta_{MLE}$, which probability should be the highest?

C should be highest
of the four.

1. Determine objective function (write)

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$

$$P(Y = 1 | \mathbf{X} = \mathbf{x}) = \sigma\left(\sum_{j=0}^m \theta_j x_j\right) = \sigma(\theta^T \mathbf{x})$$

1. What is a differentiable expression for $P(Y = y | \mathbf{X} = \mathbf{x})$?

$$P(Y = y | \mathbf{X} = \mathbf{x}) = \begin{cases} \sigma(\theta^T \mathbf{x}) & \text{if } y = 1 \\ 1 - \sigma(\theta^T \mathbf{x}) & \text{if } y = 0 \end{cases}$$

Recall remember for $B \sim \text{Ber}(p)$

Bernoulli $p(B=b) = \begin{cases} p & b=1 \\ 1-p & b=0 \end{cases} \Rightarrow p(B=b) = p^b (1-p)^{1-b}$

MLE!

$$\hookrightarrow p(y|\hat{\mathbf{x}}) = \sigma(\theta^T \hat{\mathbf{x}})^y (1 - \sigma(\theta^T \hat{\mathbf{x}}))^{1-y}$$

this, as ugly as it is, is differentiable! :)

2. What is a differentiable expression for $LL(\theta)$, log conditional likelihood?

$$LL(\theta) = \log \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta)$$

$$\begin{aligned} & \sum_{i=1}^n \log [\sigma(\theta^T \mathbf{x}^{(i)})^{y^{(i)}} (1 - \sigma(\theta^T \mathbf{x}^{(i)}))^{1-y^{(i)}}] \\ &= \sum_{i=1}^n y^{(i)} \log \sigma(\theta^T \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(\theta^T \mathbf{x}^{(i)})) \end{aligned}$$

Lisa Yan, Chris Piech, Mehran Sahami, and Jerry Cain, CS109, Spring 2024



1. Determine objective function (write)

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$
$$P(Y = 1 | \mathbf{X} = \mathbf{x}) = \sigma\left(\sum_{j=0}^m \theta_j x_j\right) = \sigma(\theta^T \mathbf{x})$$

1. What is a differentiable expression for $P(Y = y | \mathbf{X} = \mathbf{x})$?

$$P(Y = y | \mathbf{X} = \mathbf{x}) = (\sigma(\theta^T \mathbf{x}))^y (1 - \sigma(\theta^T \mathbf{x}))^{1-y}$$

2. What is a differentiable expression for $LL(\theta)$, log conditional likelihood?

$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \sigma(\theta^T \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(\theta^T \mathbf{x}^{(i)}))$$

2. Find gradient with respect to θ

Optimization
problem:

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$
$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \sigma(\theta^T \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(\theta^T \mathbf{x}^{(i)}))$$

Gradient wrt θ_j , for $j = 0, 1, \dots, m$:

$$\frac{\partial LL(\theta)}{\partial \theta_j} = \sum_{i=1}^n [y^{(i)} - \sigma(\theta^T \mathbf{x}^{(i)})] x_j^{(i)}$$

🦄 presented here without proof, though it is 🦄
a generalization of the gradient of $\mathbf{a}$ in the
 $Y = \mathbf{a}X + b + Z$ derivation from last lecture

How do we interpret the gradient
contribution of the i^{th} training datapoint?



2. Find gradient with respect to θ

Optimization
problem:

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$
$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \sigma(\theta^T \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(\theta^T \mathbf{x}^{(i)}))$$

Gradient wrt θ_j , for $j = 0, 1, \dots, m$:

$$\frac{\partial LL(\theta)}{\partial \theta_j} = \sum_{i=1}^n [y^{(i)} - \sigma(\theta^T \mathbf{x}^{(i)})] x_j^{(i)}$$

↑
scale by j-th feature

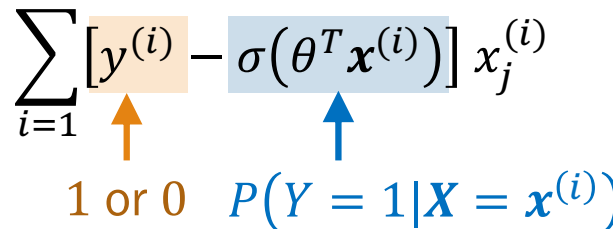
2. Find gradient with respect to θ

Optimization
problem:

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$
$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \sigma(\theta^T \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(\theta^T \mathbf{x}^{(i)}))$$

Gradient wrt θ_j , for $j = 0, 1, \dots, m$:

$$\frac{\partial LL(\theta)}{\partial \theta_j} = \sum_{i=1}^n [y^{(i)} - \sigma(\theta^T \mathbf{x}^{(i)})] x_j^{(i)}$$



2. Find gradient with respect to θ

Optimization problem:

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$

$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \sigma(\theta^T \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(\theta^T \mathbf{x}^{(i)}))$$

Gradient wrt θ_j , for $j = 0, 1, \dots, m$:

$$\frac{\partial LL(\theta)}{\partial \theta_j} = \sum_{i=1}^n \underbrace{[y^{(i)} - \sigma(\theta^T \mathbf{x}^{(i)})]} x_j^{(i)}$$

Suppose $y^{(i)} = 1$ (the true class label for the i^{th} datapoint):

- If $\sigma(\theta^T \mathbf{x}^{(i)}) \geq 0.5$, correct, so $y^{(i)} - \hat{y}^{(i)}$ is small, change θ_j minimally
- If $\sigma(\theta^T \mathbf{x}^{(i)}) < 0.5$, incorrect $\rightarrow$ change θ_j more

3. Solve

1. Optimization problem:

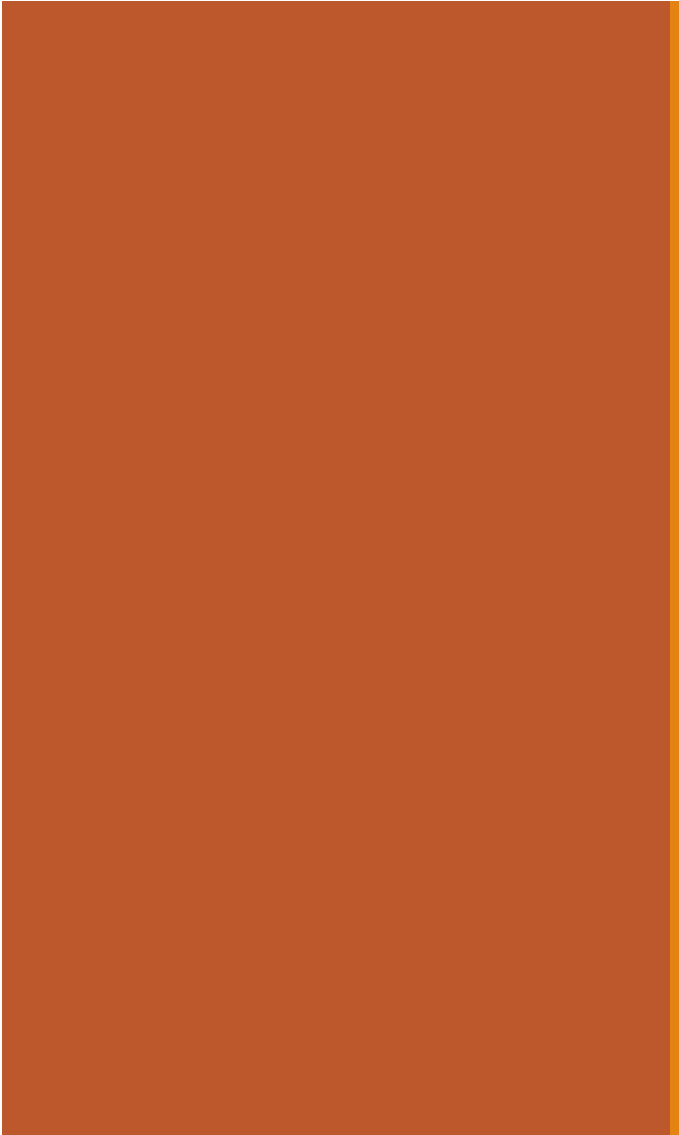
$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$

$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \sigma(\theta^T \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(\theta^T \mathbf{x}^{(i)}))$$

2. Gradient wrt θ_j , for $j = 0, 1, \dots, m$:

$$\frac{\partial LL(\theta)}{\partial \theta_j} = \sum_{i=1}^n [y^{(i)} - \sigma(\theta^T \mathbf{x}^{(i)})] x_j^{(i)}$$

3. Solve using gradient ascent!



Training: The details

Training: Gradient ascent step

$$\frac{\partial LL(\theta)}{\partial \theta_j} = \sum_{i=1}^n [y^{(i)} - \sigma(\theta^T \mathbf{x}^{(i)})] x_j^{(i)} \quad \text{for } j = 0, 1, \dots, m$$

repeat until convergence:

for all thetas:

$$\begin{aligned} \theta_j^{\text{new}} &= \theta_j^{\text{old}} + \eta \cdot \frac{\partial LL(\theta^{\text{old}})}{\partial \theta_j^{\text{old}}} \\ &= \theta_j^{\text{old}} + \eta \cdot \sum_{i=1}^n [y^{(i)} - \sigma(\theta^{\text{old}^T} \mathbf{x}^{(i)})] x_j^{(i)} \end{aligned}$$

What does
this look like
in code?

Training: Gradient Ascent

$$\text{Gradient Ascent Step} \quad \text{for } j = 0, 1, \dots, m: \quad \theta_j^{\text{new}} = \theta_j^{\text{old}} + \eta \cdot \sum_{i=1}^n [y^{(i)} - \sigma(\theta^{\text{old}^T} \mathbf{x}^{(i)})] x_j^{(i)}$$

initialize $\theta_j = 0$ for $0 \leq j \leq m$ *e.g., for $m=3$, $\theta = [0, 0, 0, 0]$*
repeat until convergence:

gradient[j] = 0 for $0 \leq j \leq m$ *e.g., for $m=3$, gradient = [0, 0, 0, 0]*

// TODO: your code here

// compute all gradient[j]'s

// based on n training examples

$\theta_j \ += \ \eta \ * \ \text{gradient}[j]$ for all $0 \leq j \leq m$

Training: Gradient Ascent

inner loop



for $j = 0, 1, \dots, m$:

Gradient
Ascent Step

$$\theta_j^{\text{new}} = \theta_j^{\text{old}} + \eta \cdot \sum_{i=1}^n \underbrace{\left[y^{(i)} - \sigma(\theta^{\text{old}^T} \mathbf{x}^{(i)}) \right]}_{\text{compute}} x_j^{(i)}$$



outer loop

compute

```
initialize  $\theta_j = 0$  for  $0 \leq j \leq m$   
repeat until convergence:
```

```
  gradient[j] = 0 for  $0 \leq j \leq m$ 
```

```
  for each training example  $(x, y)$ :
```

```
    for each  $0 \leq j \leq m$ : ← for each input feature of current training example
```

```
      // update gradient[j] for  
      // current  $(x, y)$  example
```

```
   $\theta_j \ += \ \eta \ * \ \text{gradient}[j]$  for all  $0 \leq j \leq m$ 
```

Training: Gradient Ascent

inner loop

Gradient Ascent Step

$$\theta_j^{\text{new}} = \theta_j^{\text{old}} + \eta \cdot \sum_{i=1}^n \underbrace{\left[y^{(i)} - \sigma(\theta^{\text{old}^T} \mathbf{x}^{(i)}) \right]}_{\text{compute}} x_j^{(i)}$$

outer loop

initialize $\theta_j = 0$ for $0 \leq j \leq m$
repeat until convergence:

gradient[j] = 0 for $0 \leq j \leq m$

for each training example (x, y) :

for each $0 \leq j \leq m$:

$$\text{gradient}[j] += \left[y - \frac{1}{1 + e^{-\theta^T x}} \right] x_j$$

$\theta_j += \eta * \text{gradient}[j]$ for all $0 \leq j \leq m$

Some important details...

Training: Gradient Ascent

Gradient Ascent Step

$$\theta_j^{\text{new}} = \theta_j^{\text{old}} + \eta \cdot \sum_{i=1}^n \left[y^{(i)} - \sigma(\theta^{\text{old}T} \mathbf{x}^{(i)}) \right] x_j^{(i)}$$

initialize $\theta_j = 0$ for $0 \leq j \leq m$
repeat until convergence:

gradient[j] = 0 for $0 \leq j \leq m$

for each training example (x, y) :

for each $0 \leq j \leq m$:

$$\text{gradient}[j] += \left[y - \frac{1}{1 + e^{-\theta^T x}} \right] x_j$$

$\theta_j += \eta * \text{gradient}[j]$ for all $0 \leq j \leq m$

- Finish computing gradient with θ^{old} prior to any θ update

Training: Gradient Ascent

$$\text{Gradient Ascent Step } \theta_j^{\text{new}} = \theta_j^{\text{old}} + \eta \cdot \sum_{i=1}^n [y^{(i)} - \sigma(\theta^{\text{old}^T} \mathbf{x}^{(i)})] x_j^{(i)}$$

initialize $\theta_j = 0$ for $0 \leq j \leq m$
repeat until convergence:

gradient[j] = 0 for $0 \leq j \leq m$

for each training example (x, y) :

for each $0 \leq j \leq m$:

$$\text{gradient}[j] += \left[y - \frac{1}{1 + e^{-\theta^T x}} \right] x_j$$

$\theta_j += \eta * \text{gradient}[j]$ for all $0 \leq j \leq m$

- Finish computing gradient with θ^{old} prior to any θ update
- Learning rate η is a constant you set before training

Training: Gradient Ascent

Gradient Ascent Step

$$\theta_j^{\text{new}} = \theta_j^{\text{old}} + \eta \cdot \sum_{i=1}^n [y^{(i)} - \sigma(\theta^{\text{old}^T} \mathbf{x}^{(i)})] x_j^{(i)}$$

initialize $\theta_j = 0$ for $0 \leq j \leq m$
repeat until convergence:

gradient[j] = 0 for $0 \leq j \leq m$

for each training example $(\mathbf{x}, y)$:

for each $0 \leq j \leq m$:

gradient[j] += $\left[y - \frac{1}{1 + e^{-\theta^T \mathbf{x}}} \right] x_j$



θ_j += η * gradient[j] for all $0 \leq j \leq m$

- Finish computing gradient with θ^{old} prior to any θ update
- Learning rate η is a constant you set before training
- x_j is the j^{th} feature of input $\mathbf{x} = (x_1, \dots, x_m)$

Training: Gradient Ascent

$$\text{Gradient Ascent Step } \theta_j^{\text{new}} = \theta_j^{\text{old}} + \eta \cdot \sum_{i=1}^n [y^{(i)} - \sigma(\theta^{\text{old}^T} \mathbf{x}^{(i)})] x_j^{(i)}$$

initialize $\theta_j = 0$ for $0 \leq j \leq m$
repeat until convergence:

gradient[j] = 0 for $0 \leq j \leq m$

for each training example $(\mathbf{x}, y)$:

for each $0 \leq j \leq m$:

$$\text{gradient}[j] += \left[y - \frac{1}{1 + \underbrace{e^{-\theta^T \mathbf{x}}}_{\text{sigmoid}}} \right] x_j$$


$\theta_j += \eta * \text{gradient}[j]$ for all $0 \leq j \leq m$

- Finish computing gradient with θ^{old} prior to any θ update
- Learning rate η is a constant you set before training
- x_j is the j^{th} feature of input $\mathbf{x} = (x_1, \dots, x_m)$
- Insert $x_0 = 1$ before training makes sigmoid of matrix compact!

Training: Gradient Ascent

$$\text{Gradient Ascent Step } \theta_j^{\text{new}} = \theta_j^{\text{old}} + \eta \cdot \sum_{i=1}^n [y^{(i)} - \sigma(\theta^{\text{old}^T} \mathbf{x}^{(i)})] x_j^{(i)}$$

initialize $\theta_j = 0$ for $0 \leq j \leq m$
repeat until convergence:

gradient[j] = 0 for $0 \leq j \leq m$

for each training example $(\mathbf{x}, y)$:

for each $0 \leq j \leq m$:

$$\text{gradient}[j] += \left[y - \frac{1}{1 + e^{-\theta^T \mathbf{x}}} \right] x_j$$

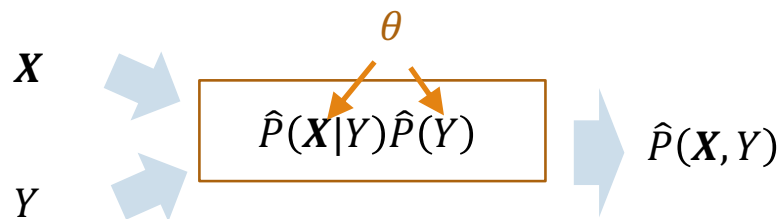
$\theta_j += \eta * \text{gradient}[j]$ for all $0 \leq j \leq m$

- Finish computing gradient with θ^{old} prior to any θ update
- Learning rate η is a constant you set before training
- x_j is the j^{th} feature of input $\mathbf{x} = (x_1, \dots, x_m)$
- Insert $x_0 = 1$ before training makes sigmoid of matrix compact! 

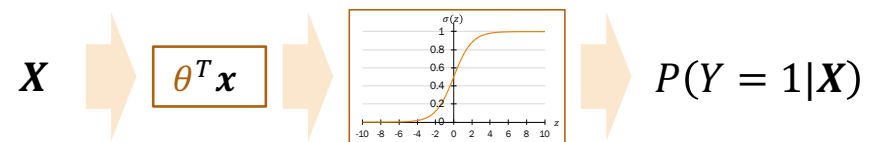
Naïve Bayes

vs

Logistic Regression



$$\hat{Y} = \arg \max_{y=\{0,1\}} P(Y | X) = \arg \max_{y=\{0,1\}} P(X|Y)P(Y)$$



$$\hat{Y} = \arg \max_{y=\{0,1\}} P(Y|X)$$

Compare/contrast:

1. What **distributions** are we modeling?
2. After learning our parameters, could we randomly **generate** a new datapoint (x, y) ?
3. Could we model a **continuous** X_j feature (e.g., $X_j \sim \text{Normal}$, or $X_j \sim \text{Unknown}$)?
4. Could we model a non-binary **discrete** X_j (e.g., $X_j \in \{1, 2, \dots, 6\}$)?



Tradeoffs:

Naïve Bayes

Logistic Regression

1. Modeling goal

$$P(\mathbf{X}, Y)$$

$$P(Y|\mathbf{X})$$

2. Generative or discriminative?

Generative: could use joint distribution to generate new points (⚠️ but you might not need this extra effort)

Discriminative: just tries to discriminate $y = 0$ vs $y = 1$ (❌ cannot generate new points b/c no $P(\mathbf{X}, Y)$)

3. Continuous input features

⚠️ Needs parametric form (e.g., Gaussian) or discretized buckets (for multinomial features)

✅ Yes, easily

4. Discrete input features

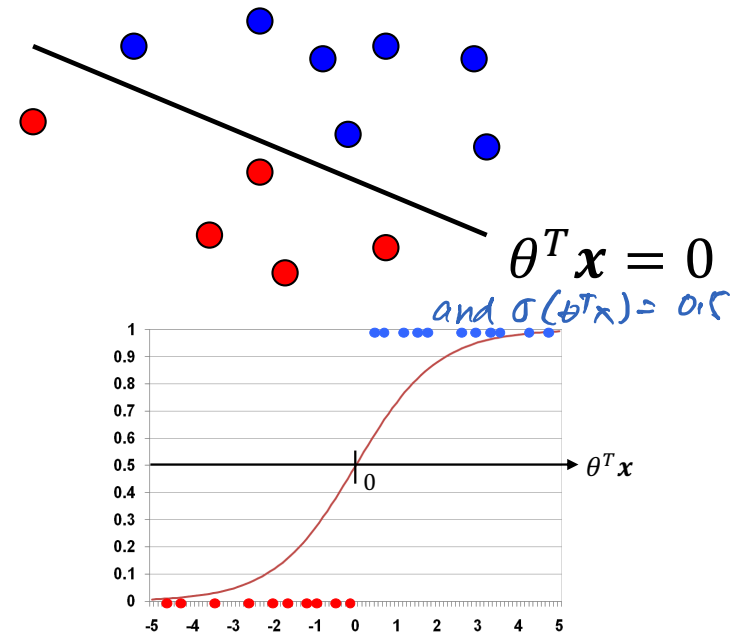
✅ Yes, multi-value discrete data = multinomial $P(X_i|Y)$

⚠️ Multi-valued discrete data hard (e.g., if $X_i \in \{A, B, C\}$, not necessarily good to encode as $\{1, 2, 3\}$)

Linearly separable data

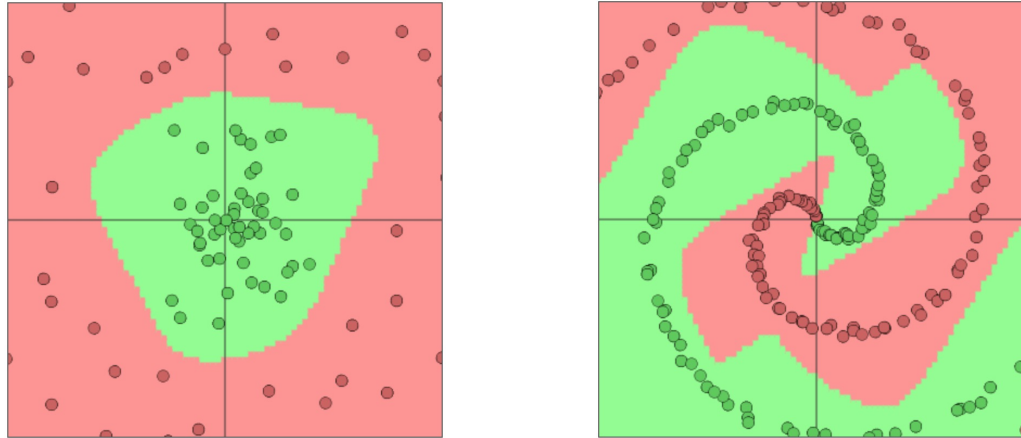
Logistic Regression is trying to find the line that separates data instances where $y = 1$ from those where $y = 0$:

- We call such data (or functions generating that data) linearly separable.
- Naïve Bayes is linear too, because there is one parameter for each feature (and no parameters that involve multiple features).

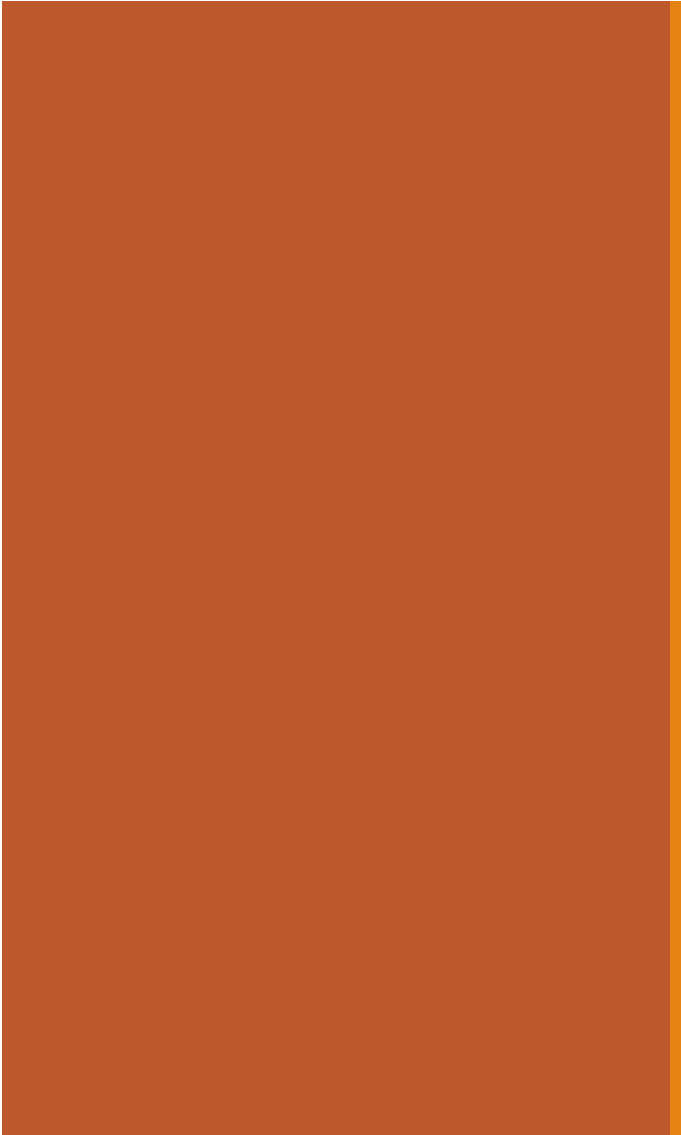


$$\hat{P}(X|Y) = \prod_{j=1}^m \hat{P}(X_j|Y)$$

Data is not always linearly separable



- Not possible to draw a line that **successfully** separates all the $y = 1$ points (green) from the $y = 0$ points (red)
- Despite this, Logistic Regression and Naive Bayes still often work well in practice



Extra: Gradient Derivation

Background: Calculus

Calculus refresher #1:

Derivative(sum) =
sum(derivative)

$$\frac{\partial}{\partial x} \sum_{i=1}^n f_i(x) = \sum_{i=1}^n \frac{\partial f_i(x)}{\partial x}$$

Calculus refresher #2:

Chain rule ★★ ★

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f(z)}{\partial z} \frac{\partial z}{\partial x}$$

Calculus Chain Rule

$$f(x) = f(z(x))$$

aka decomposition
of composed functions

Our goal

Find: $\frac{\partial LL(\theta)}{\partial \theta_j}$ where

$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \sigma(\theta^T \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(\theta^T \mathbf{x}^{(i)}))$$

log conditional likelihood

Two "pre-processing" steps to prepare for chain rule

1. Rewrite $LL(\theta)$ with $\hat{y}$
2. Compute gradient of $\hat{y}$

1. Rewriting $LL(\theta)$ with $\hat{y}$

Find: $\frac{\partial LL(\theta)}{\partial \theta_j}$ where

$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \sigma(\theta^T \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\theta^T \mathbf{x}^{(i)})) \quad \text{log conditional likelihood}$$



$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

$$\text{Let } \hat{y}^{(i)} = \sigma(\theta^T \mathbf{x}^{(i)})$$

2. Compute gradient of $\hat{y} = \sigma(\theta^T \mathbf{x})$

Aside: sigmoid has a beautiful derivative!

Sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Derivative:

$$\frac{d}{dz} \sigma(z) = \sigma(z)[1 - \sigma(z)]$$

2. Compute gradient of $\hat{y} = \sigma(\theta^T \mathbf{x})$

Sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Derivative:

$$\frac{d}{dz} \sigma(z) = \sigma(z)[1 - \sigma(z)]$$

What is $\frac{\partial}{\partial \theta_j} \hat{y} = \frac{\partial}{\partial \theta_j} \sigma(\theta^T \mathbf{x})$?

- A. $\sigma(x_j)[1 - \sigma(x_j)]x_j$
- B. $\sigma(\theta^T \mathbf{x})[1 - \sigma(\theta^T \mathbf{x})]\mathbf{x}$
- C. $\sigma(\theta^T \mathbf{x})[1 - \sigma(\theta^T \mathbf{x})]x_j$
- D. $\sigma(\theta^T \mathbf{x})x_j[1 - \sigma(\theta^T \mathbf{x})x_j]$
- E. None/other



2. Compute gradient of $\hat{y} = \sigma(\theta^T \mathbf{x})$

Sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Derivative:

$$\frac{d}{dz} \sigma(z) = \sigma(z)[1 - \sigma(z)]$$

What is $\frac{\partial}{\partial \theta_j} \sigma(\theta^T \mathbf{x})$?

$$\text{Let } z = \theta^T \mathbf{x} = \sum_{k=0}^m \theta_k x_k.$$

- A. $\sigma(x_j)[1 - \sigma(x_j)]x_j$
- B. $\sigma(\theta^T \mathbf{x})[1 - \sigma(\theta^T \mathbf{x})]x_j$
- C. $\sigma(\theta^T \mathbf{x})[1 - \sigma(\theta^T \mathbf{x})]x_j$**
- D. $\sigma(\theta^T \mathbf{x})x_j[1 - \sigma(\theta^T \mathbf{x})x_j]$
- E. None/other

$$\begin{aligned} \frac{\partial}{\partial \theta_j} \sigma(\theta^T \mathbf{x}) &= \frac{\partial}{\partial z} \sigma(z) \cdot \frac{\partial z}{\partial \theta_j} \quad (\text{Chain Rule}) \\ &= \sigma(\theta^T \mathbf{x})[1 - \sigma(\theta^T \mathbf{x})]x_j \end{aligned}$$

Compute gradient of log conditional likelihood

$$\begin{aligned}\frac{\partial LL(\theta)}{\partial \theta_j} &= \sum_{i=1}^n \frac{\partial}{\partial \theta_j} [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] && \text{Let } \hat{y}^{(i)} = \sigma(\theta^T \mathbf{x}^{(i)}) \\ &= \sum_{i=1}^n \frac{\partial}{\partial \hat{y}^{(i)}} [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] \cdot \frac{\partial \hat{y}^{(i)}}{\partial \theta_j} && \text{(Chain Rule)} \\ &= \sum_{i=1}^n \left[y^{(i)} \frac{1}{\hat{y}^{(i)}} - (1 - y^{(i)}) \frac{1}{1 - \hat{y}^{(i)}} \right] \cdot \hat{y}^{(i)} (1 - \hat{y}^{(i)}) x_j^{(i)} && \text{(calculus)} \\ &= \sum_{i=1}^n [y^{(i)} - \hat{y}^{(i)}] x_j^{(i)} = \sum_{i=1}^n [y^{(i)} - \sigma(\theta^T \mathbf{x}^{(i)})] x_j^{(i)} && \text{(simplify)}\end{aligned}$$



Compute gradient of log conditional likelihood

$$\begin{aligned}\frac{\partial LL(\theta)}{\partial \theta_j} &= \sum_{i=1}^n \frac{\partial}{\partial \theta_j} [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] && \text{Let } \hat{y}^{(i)} = \sigma(\theta^T \mathbf{x}^{(i)}) \\ &= \sum_{i=1}^n \frac{\partial}{\partial \hat{y}^{(i)}} [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] \cdot \frac{\partial \hat{y}^{(i)}}{\partial \theta_j} && \text{(Chain Rule)} \\ &= \sum_{i=1}^n \left[y^{(i)} \frac{1}{\hat{y}^{(i)}} - (1 - y^{(i)}) \frac{1}{1 - \hat{y}^{(i)}} \right] \cdot \hat{y}^{(i)} (1 - \hat{y}^{(i)}) x_j^{(i)} && \text{(calculus)} \\ &= \sum_{i=1}^n [y^{(i)} - \hat{y}^{(i)}] x_j^{(i)} && \text{(simplify)} \\ &= \sum_{i=1}^n [y^{(i)} - \sigma(\theta^T \mathbf{x}^{(i)})] x_j^{(i)}\end{aligned}$$

